Lab 5 – Code Review Template

Reviewer Name: ȘOIMAN VASILE-CRISTIAN
Reviewer Group: 217
Initial C application was created by (Name/Group): COTOC IOAN-ALIN / 212

---

0. Strengths & Positive Aspects

Before diving into areas of improvement, it is important to highlight the strong points of this implementation:

- **Basic Layered Structure:** The code is logically split into Domain (`domain.c`), Repository (`repo.c`), and Service (`service.c`) files, showing a basic understanding of architectural separation.
- **Proactive Memory Deallocation:** Destructor functions (`distruge_materie`, `distruge_repo`, `distruge_service`) are present and actively used to prevent massive memory leaks.
- **Automated Testing:** The inclusion of `ruleaza_teste()` using `assert` shows a good testing mindset, verifying core CRUD operations before the menu starts.

---

1. Functionality

List of application functionalities that are missing, are incomplete or buggy:

- **Bug (Type Mismatch / Data Loss):** In `domain.h`, the struct `MateriePrima` defines `cantitate` as a `float`. However, the constructor `creeaza_materie` and the Service functions take `cantitate` as an `int`. This causes implicit casting and data loss.
- **Bug (Undefined Behavior in UI):** In `main.c`, the application attempts to print the `cantitate` field (a `float`) using the `%d` integer format specifier (e.g., `printf("%s | %s | %d\n", ...)`). This results in printing garbage memory values.
- **Unimplemented Functions:** The code declares functions in the header files that are **never implemented** in the `.c` files:
  - In `repo.h`: `MateriePrima* cauta(Repo* r, char* nume);` is missing from `repo.c`.
  - In `service.h`: `Repo* filtrare(Service* s, char litera, int cantitate_max);` is missing from `service.c`.

2. Testing & Coverage

List of program functions (except UI) that are not covered by tests:

- **Poor Test Architecture:** Tests are written inside a single massive function (`ruleaza_teste()`) in `main.c`. Tests should be separated into modular functions (e.g., `test_repo()`, `test_service()`) and moved to a dedicated testing file.
- **Incomplete Code Coverage:** The function `copiaza_materie` in the domain module is implemented but never called in the actual code or the tests, resulting in dead code.

3. Specifications

List of program functions lacking specification:

- **Complete Lack of Specifications:** None of the functions exposed in the header files (`domain.h`, `repo.h`, `service.h`) are documented. There are no Doxygen-style comments explaining the parameters, return values, preconditions, or postconditions.

4. Modularity and Separation

Is there a clear separation between each module's specification and its implementation? Please detail…

- **No, there is a severe breach of encapsulation.** The `main.c` file (both in tests and UI) directly accesses the internal fields of the `Repository` struct (e.g., `r->lungime`, `r->elemente[0]->cantitate`, `r->capacitate`). The internal representation of the Repository must be hidden, and data should only be accessed through getter functions.
- **Business Logic Leaking into UI:** Because the `filtrare` function was never implemented in the Service, the UI takes over. In `main.c`, `ui_filtrare` iterates directly through `s->repo->elemente`. Filtering is a business rule and MUST belong in the Service.

5. Layered Architecture

Is the application correctly layered? Please detail…

- **No, the Presentation Layer is mixed with the Application Entry Point.** There is no dedicated UI module. User interaction (`printf`, `scanf`, menus) is mixed inside `main.c`. A dedicated `ui.c` and `ui.h` module must be created.
- **Bypassing the Service Layer:** In `main.c`, the sorting option calls `sortare(s->repo, 1, ordine);`. It passes the Repository directly, bypassing the Service layer entirely. Furthermore, the `sortare` function in `service.c` takes a `Repo*` as a parameter instead of a `Service*`, which breaks the standard Service pattern.

6. Memory Management

Does the application handle memory correctly (all memory is deallocated, no dangling pointers etc.). Please detail…

- **No, missing NULL checks.** The application blindly assumes memory allocations always succeed. When calling `malloc`, `realloc`, or `strdup`, the returned pointer is never checked against `NULL`.
- **Dangerous Realloc:** In `asigura_capacitate`, doing `r->elemente = realloc(r->elemente, ...)` is dangerous. If `realloc` fails and returns `NULL`, the original pointer to the elements array is lost, resulting in an unrecoverable memory leak.

7. Code Clarity

List those functions that are unclear to you (you don't understand their role or how they work – usually a sign of bad design):

- **Poor Variable Naming:** The code relies heavily on single-letter variables (`r` for Repo, `s` for Service, `m` for Materie, `c` for cantitate). This reduces readability.
- **Magic Numbers:** Functions return `1` for success, `0` for failure, and `-1` for validation errors instead of using descriptive enumerations (e.g., `enum ErrorCode`), making the function contracts unclear.

8. Action Plan & Estimation

List the problems present in the reviewed application and indicate the number of man-hours you require for fixing them:

Problem	Required Fix	Estimated Time
Type Inconsistencies	Standardize `cantitate` as `float` across Domain, Repo, Service, and UI function parameters.	0.5 hr
Print Formatting Bug	Change `%d` to `%f` or `%.2f` in `main.c` UI functions.	0.2 hr
Missing Implementations	Implement `cauta` in `repo.c` and `filtrare` in `service.c`.	0.5 hr
Encapsulation Breakdown	Create getters for `Repo` (`get_len`, `get_elem`) and replace all direct `r->` accesses in `main.c` and `service.c`.	1.0 hr
Layer Violations (UI & Filtering)	Extract UI into `ui.c`/`ui.h`. Move filtering logic into the	1.0 hr

Problem	Required Fix	Estimated Time
	Service layer (return a dynamic array to the UI).	
Layer Violations (Sorting)	Change `sortare` signature to accept `Service* s`. Update UI to call it via the Service, not the Repo.	0.5 hr
Memory Allocation Vulnerabilities	Add `if (ptr == NULL)` checks for all allocations. Fix the `realloc` pointer reassignment bug.	0.5 hr
Missing Specifications	Add Doxygen-style documentation (param, return, pre/post conditions) to all header files.	0.5 hr
Test Restructuring	Extract tests into modular functions in a separate `test.c` file.	0.5 hr
Code Clarity	Refactor magic numbers to `enum` and rename short variables for clarity.	0.5 hr
Total Estimated Time		**~5.7 hrs**